

서민재

BE Engineering · Reliability · Search & Data · AI Engineering

Java와 Kotlin으로 이벤트·배치·검색·캐시 시스템을 설계하고 운영해 온 백엔드 엔지니어입니다. 장애를 한 번 처리하고 끝내기보다 상태와 재시작 지점을 코드에 남겨, 같은 문제를 다시 추적할 수 있게 만드는 데 관심이 많습니다. 최근에는 코드 탐색과 설계 검토, 테스트 절차를 에이전트와 스킴으로 구조화하고 AI가 만든 결과도 테스트와 실행 기록으로 다시 확인합니다.

ckdekrn88@gmail.com

github.com/ch4570

velog.io/@ch4570

linkedin.com/in/devseorex

이력서

포트폴리오

Seoul, Korea

경력 요약

01 · RELIABILITY

실패한 위치와 다시 시작할 기준을 남깁니다.

이벤트 상태, 수동 복구 전략, 배치 페이지와 이관 순서를 코드와 테스트로 관리했습니다.

02 · SEARCH & DATA

빠른 경로와 정확한 경로의 경계를 나눕니다.

후보 수집·랭킹·캐시 계층을 분리하고 요청 조건이 달라질 때만 필요한 범위를 다시 계산했습니다.

03 · AI-NATIVE

개인의 판단을 다시 실행할 수 있는 개발 자산으로 만듭니다.

에이전트와 스킴, 설치 도구, 평가 계약을 함께 관리하고 자동 변경에는 사람의 승인 단계를 뒀습니다.

9종 · 6상태 · 8전략

포인트 이벤트와 15분 정체 알림

10개 테이블

원장 아카이브 이관 순서

9개 소스 · 500→100→20

검색 후보와 랭킹 단계

29개 · 43개

공통 테스트 도우미와 검증 테스트

2025.06 — 현재

웍스피어(유)

JOBKOREA / Albamon Platform · Backend Engineer

2024.06 — 2025.04

토스랩

JANDI · Backend Engineer

2023.09 — 2024.05

오케스트로

IDP / Vista · Backend Engineer

2022.11 — 2023.02

즐거운

Backend Engineer

웍스피어(유) · Backend Engineer

JOBKOREA·Albamon Platform에서 서비스 기능부터 이벤트 신뢰성, 배치, 검색·캐시, 공통 테스트 기반까지 백엔드 엔지니어링 전반을 맡고 있습니다.

2025.06 — 현재

POINT PLATFORM · EVENT RELIABILITY

포인트 이벤트가 어디에서 멈췄는지 상태로 남기고, 실패 지점에 맞는 복구 경로를 만들었습니다.

메시지를 보내지 못한 경우와 소비자가 처리하다 실패한 경우는 다시 시작할 위치가 다릅니다. 운영자가 로그와 원장을 매번 대조하지 않아도 되도록 처리 단계를 이벤트 레코드에 남겼습니다.

- 포인트 내부 이벤트 9종을 생성부터 복구 불가 실패까지 6개 상태로 기록하도록 설계했습니다.
- 발행 실패와 소비 중 실패의 재시작 지점을 나누고, 저장된 payload를 이벤트별 8개 복구 전략으로 전달하는 수동 복구 API를 구현했습니다.
- 처리 중 상태가 15분을 넘긴 추적 대상은 이벤트 유형별로 모아 Slack으로 알렸습니다.
- 배치 재시도와 프록시 호출 경로를 바로잡고 포인트 선점 복구 흐름을 보강했습니다.

운영에서 달라진 점 전송 실패, 처리 실패, 장기 미처리를 따로 조회하고 멈춘 위치에 맞춰 복구를 시작할 수 있습니다.

역할	기술	범위
설계 · 구현 · 테스트	Kotlin · Spring Boot · Kafka	9 event types · 6 states · 8 strategies

POINT LEDGER · BATCH / ARCHIVE

원장 소멸은 페이지 단위로 처리하고, 10개 아카이브 테이블은 이관 순서까지 테스트했습니다.

소멸, 만료 안내, 아카이브는 같은 원장 데이터를 다루지만 시작 시점과 후속 작업이 다릅니다. 한 번에 읽고 쓰던 범위를 줄이고 각 작업이 어디까지 진행됐는지 확인할 수 있게 나눴습니다.

- 포인트 소멸 작업을 한 번에 처리하는 Tasklet에서 페이지 기반 Chunk 처리로 바꿔 조회와 쓰기 단위를 분리했습니다.
- 만료 대상 수집, 날짜 계산, 기업 회원 필터, 알림 레코드 일괄 저장, Kafka 발행 순서를 테스트로 고정했습니다.
- point_core 6개와 point_service 4개 등 10개 아카이브 테이블을 정해진 순서로 이관하는 operator 구성을 구현했습니다.
- 같은 예산의 동시 차감, 유료·무료 원장 혼합 사용, 회원 식별자 중복, lock key가 같은 요청과 다른 요청을 나눠 검증했습니다.

재시작 기준 소멸은 페이지, 만료 안내는 발송 상태, 아카이브는 테이블 순서로 처리 위치를 확인합니다.

검색·캐시와 테스트 기반

사용자가 보는 결과의 정확성과 개발자가 변경을 확인하는 방법을 함께 다뤘습니다.

FEED & SEARCH · SERVING / RETRIEVAL

후보 수집과 랭킹, 캐시 재사용과 무효화를 한 가지 흐름으로 묶지 않았습니다.

좌표나 필터, 로그인 상태가 달라지면 캐시에 값이 있어도 현재 요청에는 오래된 결과가 됩니다. 반대로 정상 스크롤까지 매번 다시 계산하면 검색과 랭킹 비용이 커집니다.

- OpenSearch 후보 소스 9개를 병렬 조회하고, 소스마다 최대 200건을 받아 1차 랭킹 500건, 2차 랭킹 100건, 기본 페이지 20건으로 좁혔습니다.
- Snapshot, 1차 랭킹 풀, 2차 랭킹 풀, 본 공고 저장소를 나눠 정상 스크롤은 가장 싼 캐시부터 사용하게 했습니다.
- 세션 캐시는 30분 동안 재사용하되 좌표·필터 요청 지문이나 로그인 주체가 달라지면 관련 후보 풀을 무효화했습니다.
- Redis 장애에서는 메모리 계산으로 우회하고, contentHash 중복 제거와 위치 기반 10km 반경 정책을 예외 경로에 반영했습니다.

판단 기준 스크롤, 후보 소진, 조건 변경, 로그인 전환, Redis 장애를 같은 cache miss로 취급하지 않았습니다.

역할	기술	범위
설계 · 구현 · 테스트	Kotlin · OpenSearch · Redis	9 sources · 500 → 100 → 20

ENGINEERING PLATFORM · TEST SUPPORT

테스트마다 되풀이하던 준비 코드를 공통 모듈로 옮겨, 테스트의 시작점을 맞췄습니다.

fixture와 mock 초기화, WebMvc 요청, JPA 데이터 준비가 테스트마다 달라지면 무엇을 검증하는지보다 setup을 읽는 데 시간이 더 듭니다. 반복되는 준비 절차를 작은 도우미로 모으고 공통 동작 자체도 테스트했습니다.

- fixture·mock·WebMvc·JPA·transaction 준비와 공통 진입점을 29개 구현 파일로 정리했습니다.
- 기본 테스트, mock 테스트, WebMvc 테스트, JPA 테스트의 진입점을 나눠 필요한 컨텍스트만 선택하게 했습니다.
- MockMvc 요청 빌더와 응답 검증, query string null 처리, JPA 초기화와 transaction 도우미를 제공했습니다.
- 공통 모듈의 동작은 43개 테스트 소스로 확인하고 느린 테스트 제외, 기본 profile, JavaDoc까지 함께 정리했습니다.

팀에서 쓰는 방식 테스트 종류에 맞는 기반 클래스를 고르면 fixture와 mock, Spring 컨텍스트 준비 규칙을 같은 방식으로 사용할 수 있습니다.

Agent Lab · 에이전트와 스킬을 설치·평가 가능한 개발 자산으로 만들었습니다.

업무에서 반복한 코드 탐색, 설계 검토, 테스트, 리뷰 절차를 특정 저장소에 묶인 프롬프트가 아니라 다른 프로젝트에도 옮길 수 있는 단위로 정리했습니다.

V0.1.0.0

역할, 지식, 절차와 평가를 나누고 manifest에서 연결했습니다.

- 15개 에이전트, 14개 지식 번들, 32개 스킬을 역할·지식·절차로 나누고 24개 agent→skill 연결과 11개 스킬 의존성을 선언했습니다.
- 선택한 자산과 연결된 의존성을 재귀적으로 찾아 Claude Code와 Codex의 공식 경로에 설치하는 shell 기반 도구를 만들었습니다.
- 설치 회귀 52/52와 Codex 스킬 탐색 32/32를 확인하고, 구조 fixture 7/7과 행동 계약 10/10을 정적으로 검증했습니다.
- NL-SQL guardian, index migration architect, search relevance evaluator, code cartographer처럼 검색·DB·코드 이해에 필요한 도구를 추가했습니다.

현재 경계 Agent Lab은 실행 런타임이 아니라 이식 가능한 자산 저장소입니다. 장기 메모리 저장소, 프롬프트 자동 수정, 스킬 자동 승격은 아직 포함하지 않았습니다.

실행 경험이 곧바로 규칙이 되지 않도록 사람의 승인 단계를 뒀습니다.

실행 로그와 테스트, 리뷰 의견에서 개선 후보를 모으되 출처와 적용 범위, 충돌, 위험을 먼저 확인합니다. 후보는 평가 기록과 되돌리기 방법을 갖춘 변경안으로 남기고, 사람이 승인한 뒤에만 다음 설치에 반영하도록 설계했습니다.

INVENTORY	INSTALL	DISCOVERY	CONTRACTS
15 agents · 32 skills	52/52 regression	Codex 32/32	Structure 7/7 · Behavior 10/10

PREVIOUS ROLE

토스랩 · JANDI Backend Engineer

2024.06 — 2025.04

커버리지 수치는 70%였지만 배포는 여전히 불안했습니다. 테스트가 실제 구현을 확인하는지, CI가 같은 조건으로 반복되는지부터 다시 봤습니다.

- Drive-Service를 Spring Boot 2.3.8에서 3.3.3으로 올리며 테스트를 0개에서 160개 이상으로 늘렸습니다.
- Mock 테스트가 통과한 뒤 운영에서 드러난 JPA 변경 감지 오류를 실제 구현 테스트로 재현하고, 보정 스크립트로 약 200만 건의 데이터를 바로잡았습니다.
- CI 메모리 부족은 실행과 컨텍스트 재사용, 초기화 구조를 바꿔 자원 증설 없이 해결했습니다.

인증·인가와 OpenSearch 조회부터 서비스 운영까지 경험했습니다.

오케스트로 · Backend Engineer

IDP / Vista

2023.09 — 2024.05

- 신용정보원 IDP에서 Spring Cloud API Gateway와 Spring Security·JWT 기반 인증·인가를 개발했습니다.
- SonarQube·Jenkins 연동 API를 구현해 프로젝트 품질과 배포 정보를 조회하도록 만들었습니다.
- 하나금융그룹 Vista에서 VM 메트릭 조회 API와 조건별 OpenSearch 동적 쿼리를 개발했습니다.

즐거운 · Backend Engineer

자사 서비스 개발·운영

2022.11 — 2023.02

자사 서비스의 백엔드 기능을 개발하고 운영 중 발생한 문제를 해결했습니다.

발표 · 기록

YouthCon 2025 Speaker

'우리팀을 위한 Spring Test 만들기' 발표

기술 글 156편

Java·JPA·테스트·SQL 튜닝 중심, 2026.07 기준

글또 10기

백엔드·인프라 빌리지 준비위원과 연사

오픈소스

Spring OSS 개선 3건 반영

Spring Kafka · Spring Boot · Spring Data JPA

OpenSearch 한글 분석 플러그인

한·영 자판, 초성·자모 검색을 OpenSearch 2.19와 Kotlin으로 구현

학력 · 자격

한국방송통신대학교

컴퓨터과학과 졸업 · 2023.03 — 2025.03

Certified Kubernetes Administrator

2024.09

SQLD

2022.06